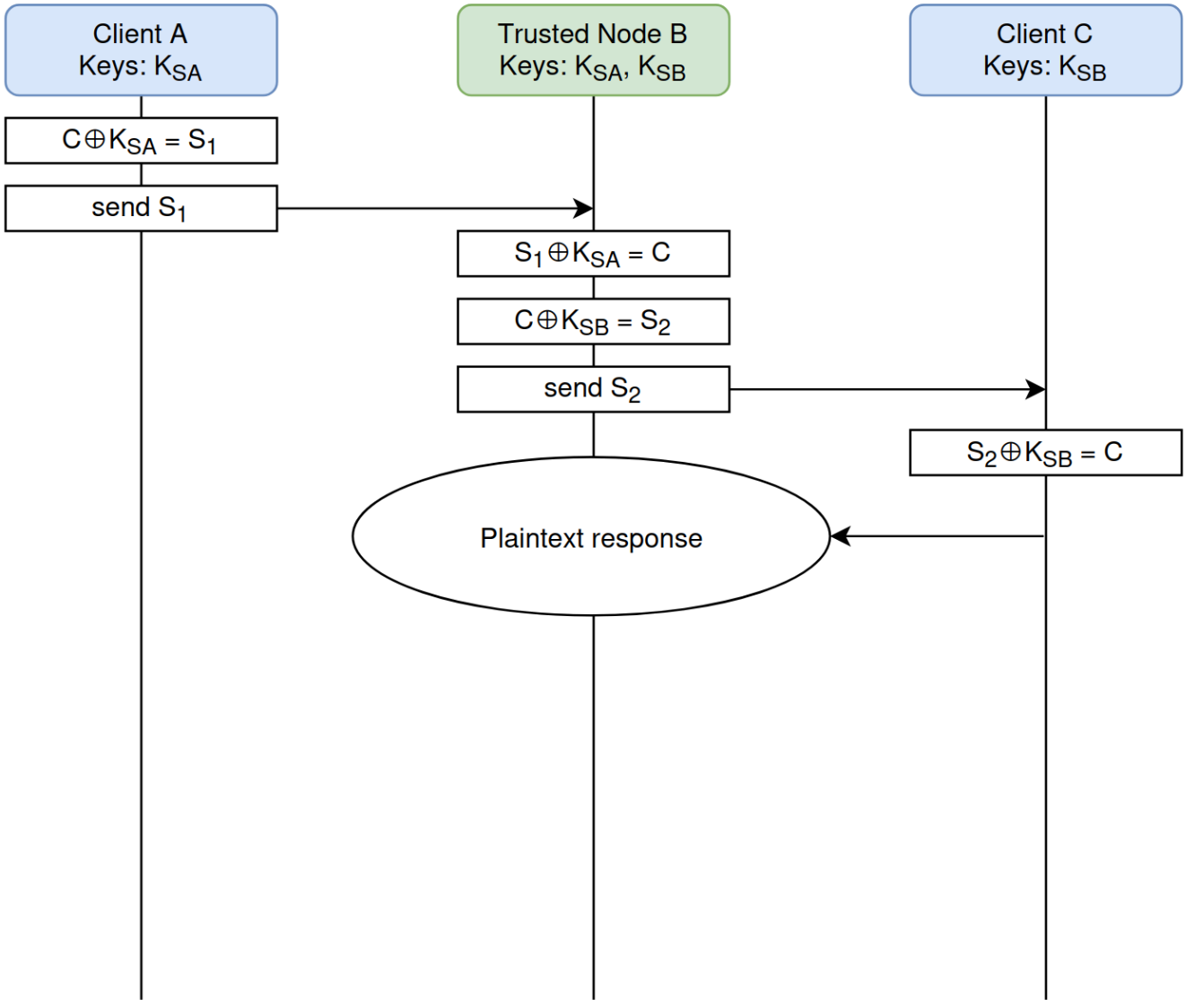
Exercise 3: Implementing the Trusted Node (Node B)

You implement Node B: the trusted relay between A and C.

Node B already has shared keys with both A and C. A initiates key relaying by sending the XORed key together with the key ID of the key that should be used to decrypt it. Then B requests a shared key with C and sends the XORed message to C, again including the corresponding key ID.



Flow:

1. Connect to your KMS, to A, and to C.
2. Wait for A to send an encrypted message.
3. Fetch the decryption key from your KMS (shared secret with A).
4. Decrypt.
5. Fetch the encryption key from your KMS (shared secret with C).
6. Re-encrypt with the new key, forward to C.

Connection details

Var	Value
KMS base URL	`http://tii-qkd-workshop.uaenorth.cloudapp.azure.com:5002/api/v1/keys/`
A's KMS id (`dec_keys`)	`K002X001`
C's KMS id (`enc_keys`)	`K002X003`
A's address (gRPC)	`tii-qkd-workshop.uaenorth.cloudapp.azure.com:50050`
C's address (gRPC)	`tii-qkd-workshop.uaenorth.cloudapp.azure.com:50060`

KMS API (ETSI 014, plain HTTP)

```
GET {base_url}/<peer_sae_id>/enc_keys
GET {base_url}/<peer_sae_id>/dec_keys?key_ID=<uuid>
```

Response: `{"keys": [{"key_ID": "...", "key": "<base64>"}]}`. Decode base64, XOR against the message body.

gRPC proto - generate your own stubs from this

```
syntax = "proto3";

package key_swap;

service Relay {
    rpc Connect(stream RelayMsg) returns (stream RelayMsg) {}
}

message Ready {}

message RelayMsg {
    oneof payload {
        Ready ready = 1;
        KeySwapMessage request = 2;
        KeySwapMessageAck reply = 3;
    }
}

message KeySwapMessage {
    string key_uuid = 1;
    bytes data = 2;
}

message KeySwapMessageAck {
    string reply = 1;
}
```

Open one `Connect` stream to A, one to C. Send `RelayMsg{ready: Ready{}}` first on each. A pushes a `request` down its stream when ready; decrypt it, fetch a fresh key for C, and send a `request` (re-encrypted) on C's stream.